
Mobile API Protection for Banking

A reference architecture for stopping API abuse from tampered, repackaged, and instrumented mobile clients — anchored in server-verified runtime attestation.

Runtime Attestation

Request-Bound Signing

Gradle + OkHttp SDK

OWASP MASVS Aligned

PSD2 · PCI

EXECUTIVE SUMMARY

What this protects — and how it stands up to a bank security review

Banking apps face a specific, well-understood problem: static analysis of a shipped APK (jadx, apktool, APKLeaks) reliably reveals every REST endpoint, request/response schema, header, and any embedded key — even under R8/ProGuard. Dynamic analysis with Frida on a rooted device goes further, reading traffic in cleartext before TLS. This is not a bug to be patched; it is a structural property of the mobile platform.

THE CLAIM WE DELIBERATELY DO NOT MAKE

We do not sell "un-reverse-engineerable" or "invisible API" hiding. That claim is false on the mobile platform and any competent bank security team will reject it. OWASP MASVS-RESILIENCE states plainly that it is *impossible to prevent static analysis*, and that obfuscation "must never be used as a replacement for security controls."

THE CLAIM WE DO MAKE — AND CAN DEFEND

We make a leaked endpoint, key, or captured request **useless to anything that is not your genuine, unmodified app running on a plausibly clean device**. Trust is anchored on **your server**, not in the binary: the app proves its integrity via hardware-backed attestation, receives only short-lived request-bound credentials, and your backend rejects everything else. This is the same architecture banks already accept under PSD2 and PCI.

<1 ms

Added latency per request (signing on the hot path; attestation amortized)

2 lines

Developer integration: one Gradle plugin, one OkHttp client factory call

Fail closed

Security-critical calls are rejected server-side without valid attestation

This brief is intentionally candid about limitations (page 6). In a banking procurement, that candor is a feature: it demonstrates the architecture is honest about where the real control lives — on the server — which is exactly what an auditor verifies.

THE PROBLEM, PRECISELY STATED

Two problems banks conflate — only one is solvable on the client

OBJECTIVE	SOLVABLE CLIENT- SIDE?	WHY
Hide endpoints / keys / payloads from an attacker	No	The app must resolve the URL and build/decrypt the payload in its own memory. An attacker who controls the device recovers both — statically from the binary, dynamically via Frida hooking the HTTP layer before TLS. Native/NDK code only raises cost by hours.
Make a leaked endpoint/key un-usable by anything but the genuine app	Yes (server-anchored)	Move the trust anchor to the server. Attestation proves "a genuine, unmodified app instance made this request"; the signing secret lives server-side; short-lived, request-bound credentials mean captured traffic cannot be replayed or forged at scale.

The product is built entirely around the second objective. The first is addressed only as a *friction layer* — worthwhile to deter commodity attackers and satisfy MASVS-RESILIENCE hardening checkboxes, never presented as the security control.

Why the honest framing wins the banking sale

Bank security reviewers, PCI QSAs, and PSD2 auditors evaluate against a threat model that *assumes* the client is hostile. A vendor promising client-side secrecy signals inexperience and fails technical due diligence. A vendor whose architecture states "assume the API is public; here is how we make abuse non-actionable and auditable" maps directly to the controls those reviewers are looking for. Candor is the differentiator, not a liability.

REFERENCE PRECEDENT

This is the same architecture shipped commercially by Approov and used in production banking apps, and the pattern Google Play Integrity and Apple App Attest are designed to feed. The differentiator of this SDK is **developer experience and a bundled server-side verifier**, not a novel secrecy claim.

Two layers, two jobs, never confused

CRYPTOGRAPHIC / ATTESTATION LAYER

Server-anchored trust. The signing secret lives on the server; the backend rejects any request lacking a fresh, valid, request-bound proof of app integrity. **This is the security control.**

OBFUSCATION LAYER

Endpoint-table obfuscation, NDK constexpr-XOR strings, control-flow hardening. Raises the cost of static analysis. **This is friction / defense-in-depth — never the control.**

Components

COMPONENT	RESPONSIBILITY
Gradle plugin build-time	Orchestrates the build on the AGP 8.x/9.x ASM Instrumentation API. Registers codegen and the NDK build, applies opt-in endpoint-string obfuscation per variant, and fails the build on foot-guns (hardcoded secrets, debuggable release, missing pinning, cleartext traffic).
KSP codegen compile-time	From an annotated Retrofit interface, generates the obfuscated endpoint table, the canonical-request serializer used for signing (method + path + sorted params + body hash + nonce), per-endpoint policy, and JWE wrappers for encrypted bodies. Zero runtime reflection.
NDK core libhidesecc.so	Runs the challenge-response attestation handshake, requests just-in-time short-lived secrets from the server after attestation passes (nothing long-lived is embedded), computes the per-request signature, and uses Android Keystore / StrongBox for device-bound keys where available.
OkHttp interceptor runtime · dev-facing	Attaches the short-lived signed attestation token (60–300 s TTL) bound to the request body hash, nonce, and user session; enforces dynamic certificate pinning (OTA pin rotation, no app release); optionally JWE-encrypts bodies; fails closed on missing attestation for critical calls.
Server-side verifier the load-bearing control	Drop-in gateway middleware (Node/Java/Go). Verifies the token signature, freshness, anti-replay nonce, and body binding; verifies Play Integrity verdicts server-side; feeds results into risk scoring alongside authN/Z, rate limiting, and anomaly detection.

Layered defense — what each layer defeats

Server enforcement

AuthN/Z · short-lived revocable tokens · rate limits · anomaly detection → the policy backstop; nothing is bypassed here

DURABLE CONTROL

Attestation (server-verified)

Play Integrity / challenge-response verdict → risk scoring · defeats repackaged apps, emulators, bots at scale · probabilistic vs. a determined bypass

SECURITY CONTROL

Cryptographic (client → server)

Per-request signature bound to body + nonce · JIT runtime secrets · dynamic pinning · defeats forged/replayed requests & passive MITM

SECURITY CONTROL

Obfuscation (client)

FRICTION ONLY

Endpoint-table obfuscation · NDK constexpr-XOR · control-flow · defeats jadx/apktool/APKLeaks & scripted attackers · not a Frida attacker

DEVELOPER EXPERIENCE

What the developer writes

The entire integration is a plugin declaration and one client factory call. Attestation handshakes, header injection, pinning, secret substitution, and canonicalization are handled by the SDK. This DX is the core product value.

```
// build.gradle.kts
plugins { id("io.hidesec.security") version "1.x" }

hidesec {
    attestationService = "playintegrity" // or "cloud"
    serverVerifierUrl  = "https://api.yourbank.com/attest"
    obfuscateEndpoints = true           // friction layer, opt-in
    failMode           = FailMode.CLOSED
}

// Retrofit interface – annotate; codegen does the rest
@AttestedApi
interface AccountApi {
    @POST("v1/transfer")
    suspend fun transfer(@Body req: TransferRequest): TransferResponse
}

// Runtime – one call; interceptor is pre-wired
val client = Hidesec.okHttpClient()
val api = Retrofit.Builder().client(client) /* ... */ .create(AccountApi::class.java)
```

PERFORMANCE

Latency budget

COST	BUDGET	HOW IT IS MET
Attestation handshake (Play Integrity standard, warm)	~200–500 ms	Amortized: fetched per session at startup, async; short-lived token cached. Never on the per-request path.
Per-request signature (software HMAC over body hash)	sub-ms	Signing key kept in software / TEE on the hot path
Interceptor overhead (header compute + substitution)	<1 ms	—
Body-inclusive signature	∝ payload	Stream-hash where possible; do not buffer large uploads
DEX obfuscation	cold-start cost	Disclosed & opt-in per variant — the friction layer can be disabled where latency-sensitive

ENGINEERING CONSTRAINT STATED UP FRONT

The "<1 ms/request" figure holds only when attestation is amortized off the request path. A per-request Play Integrity call costs hundreds of milliseconds and collides with the default 10,000-requests/day

quota. The architecture mandates async token provisioning with caching — and a defined degraded mode for non-GMS devices — precisely to keep this budget honest.

HONEST LIMITATIONS

What a determined attacker still gets — and why the server is load-bearing

Presenting these to a bank is deliberate. Each is a known property of the platform, and the architecture is designed so that none of them compromises the backend.

A ROOTED-DEVICE FRIDA ATTACKER CAN...	MITIGATION / WHY IT DOESN'T BREAK THE BACKEND
Read every endpoint, request, and response in cleartext (hook OkHttp before TLS, or strip pinning + MITM)	Obfuscation and pinning buy time, not secrecy. The backend never relies on the request being secret — it relies on the request being <i>attested and signed</i> .
Recover any decrypted secret in the moment (hook the decrypt routine / Cipher)	JIT secrets are short-lived and centrally rotatable; a captured secret expires in minutes and is revocable on leak detection.
Drive a genuine, attested app instance to mint valid tokens (the hostile-client problem)	Body binding stops blind replay; scaled abuse is caught by rate limiting, anomaly detection, and account-level risk scoring on the server.
Bypass attestation itself at a cost (keybox spoofing, bypass modules)	Attestation is one weighted risk signal, not a sole gate. Cost is raised materially; residual risk is handled by backend policy.

THE DESIGN TEST

If the client SDK vanished entirely, a correctly secured backend would still be safe. The SDK cuts abuse cost, blocks commodity and scaled attacks, and produces auditable integrity signals — but the durable controls (authenticate and authorize every request, short-lived revocable tokens, rate limiting, anomaly detection, secrets off the client) live on the server. This is exactly what a PCI QSA or PSD2 auditor expects to verify.

Compliance and operational checklist

KEY & SECRET MANAGEMENT

- No long-lived signing key in the binary — server-held, delivered JIT post-attestation
- Central rotation without an app release; short validity windows; revocation on leak
- Per-request nonce + server-side anti-replay cache

CI/CD & SUPPORTABILITY

- Headless Gradle plugin; hardening off in debug, on in release
- R8 + obfuscation mapping files archived to crash reporting for de-obfuscated stack traces
- Attestation credentials in the CI secrets manager, never in source

SUPPLY CHAIN

- Signed artifacts (Sigstore / Maven signing); published SBOM
- Pinned, maintained dependencies; reproducible NDK toolchain

AVAILABILITY (DESIGN A DEGRADED MODE)

- Play Integrity needs Google Play Services — define an explicit tier for non-GMS / sideloaded / offline
- Plan for the 10,000/day quota; request an increase before wide rollout
- Warm-up call on cold start; server-side clock-skew tolerance + retry-on-401 re-provisioning

REGULATORY MAPPING

- **PCI DSS / MPoC / SPoC:** attestation + RASP + key protection map to software-security & tamper-detection; server enforcement covers auditable controls
- **PSD2 RTS (SCA):** app hardening + attested secure channel supports transaction integrity; SCA remains a server + independent-channel control
- **OWASP MASVS:** positioned as MASVS-RESILIENCE (V8) defense-in-depth *on top of* the MASVS-L1/L2 baseline

Buy-vs-build honesty for the prospect

IF THE BANK...	RECOMMENDATION
needs the outcome now, cross-platform	The SDK, or Approov as the reference commercial equivalent; Play Integrity + App Attest if strong in-house backend capacity
needs certified code hardening for a checkbox	Bundle with DexGuard-class hardening — but only <i>on top of</i> this server-side core
wants baseline hygiene only	R8 + string obfuscation + cert pinning, with zero expectation of API secrecy

WHERE THIS SDK'S VALUE ACTUALLY IS

No existing open-source product packages endpoint obfuscation + build-time hardening + OkHttp middleware + attestation + a drop-in server verifier into one integration. The differentiator is **developer experience and a bundled verifier** — making the correct, auditable architecture a two-line integration for the bank's mobile team.

available on request.